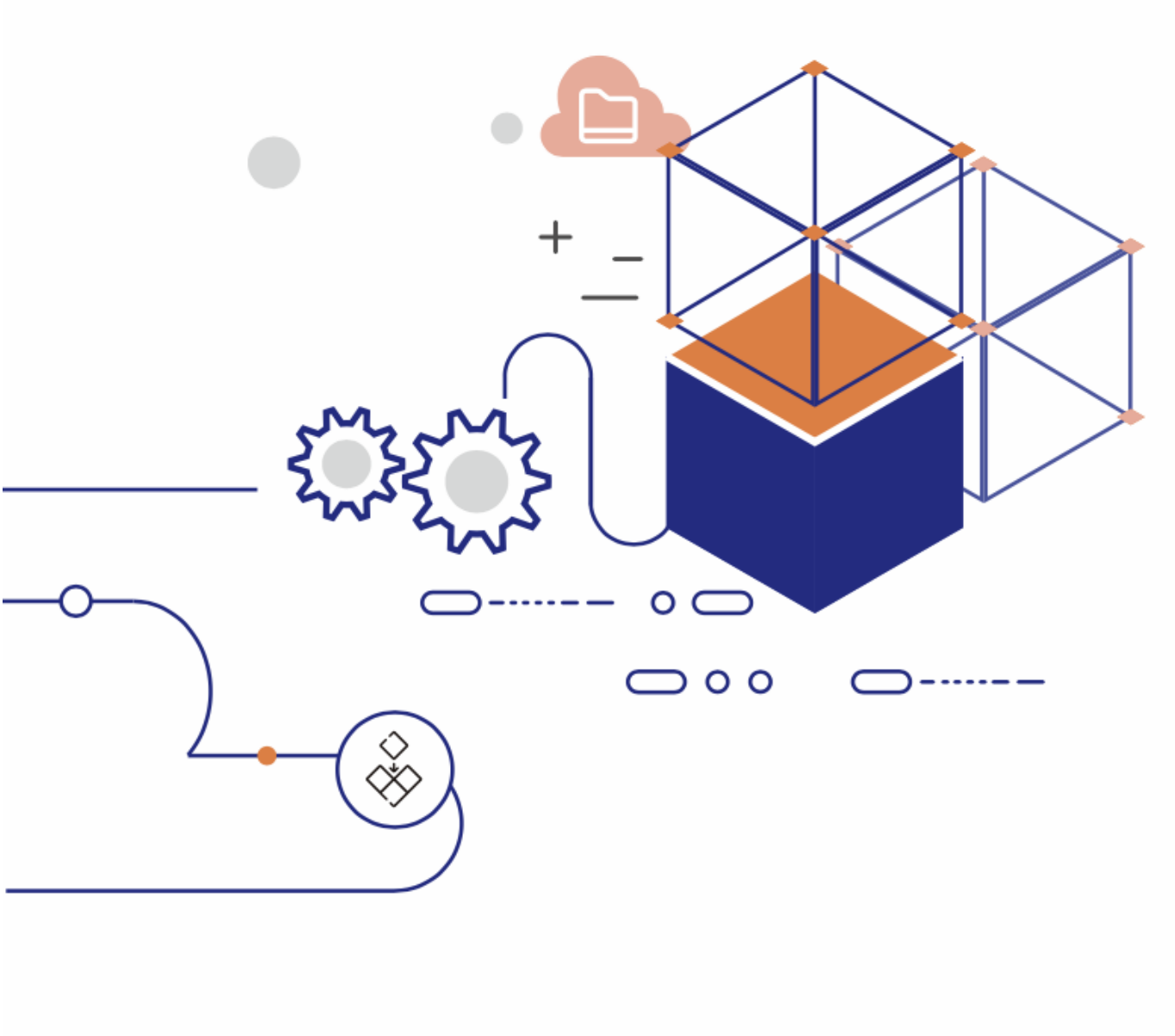
Table of Contents



EloqKV: Redis-Compatible Key-Value Store with Memory-Like Performance and SSD Economics

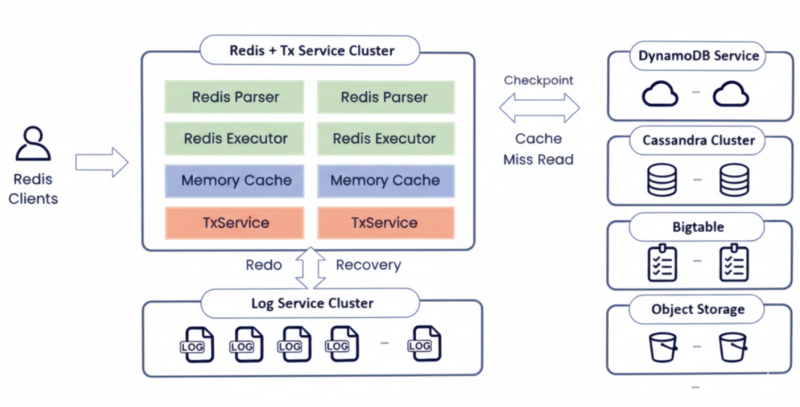
Introduction

EloqKV is a next-generation, Redis-compatible key–value database that delivers memory-class performance at SSD cost. Unlike traditional in-memory caches, EloqKV is designed and operated as a **primary, durable data store** rather than a best-effort cache. It guarantees data persistence, provides strong consistency across a distributed cluster, and integrates seamlessly with existing Redis clients and tooling—typically requiring **no application code changes**.

By replacing expensive DRAM with NVMe SSDs for the majority of the data footprint, EloqKV removes the hard capacity ceiling of DRAM-based systems while delivering up to **10x lower total cost of ownership (TCO)** for large-scale workloads.

Achitecture

EloqKV is a Redis-compatible distributed key value store powered by Data Substrate. Its architecture includes a frontend compute engine compatible with the Redis protocol. Within Data Substrate, the TxService is responsible for caching hot data and managing transaction processing, while the LogService handles data persistence. LogService replicas are distributed across different availability zones (AZs) to ensure tolerance to AZ-level failures. The underlying storage layer supports pluggable key-value (KV) storages, such as EloqStore, AWS DynamoDB, Google Bigtable, and Cassandra. These storage services store cold data for cache misses and provide high availability for baseline data.



Core Features

10x Cost Efficiency at Scale

EloqKV is optimized for standard Redis data structures and access patterns, but is architected around SSD-native data management. For memory-bound workloads in the **100 GB–100-TB** range per cluster, EloqKV:

- **Offloads cold and warm data to NVMe SSDs** while keeping hot working sets in DRAM.

- **Preserves Redis-like latency characteristics**, even when the total dataset is far larger than memory.
- **Reduces infrastructure footprint by up to 10x**, often replacing large Redis clusters with a small number of NVMe-optimized nodes.

This allows teams to scale capacity based on SSD pricing, not DRAM constraints.

Primary-Database Durability and Consistency

EloqKV is engineered as a database-grade system, not just a cache:

- **Write-Ahead Logging (WAL)** ensures durable, crash-safe persistence for every write.
- **No cache-invalidation complexity**: applications read and write to a single source of truth rather than a cache layered on top of another database.
- **Native distributed execution** supports multi-key transactions (`MULTI/EXEC`) and Lua scripts across shards—capabilities that are difficult or unsafe to achieve with standard Redis deployments.
- **Cluster-wide transactional semantics** maintain consistent state for cross-node transactions, enabling EloqKV to back mission-critical online systems.

Latency-Optimized and Stable Under Pressure

Conventional Redis clusters often experience latency spikes during background operations (fork-based snapshots, large RDB generation, or replica resync) and hit practical limits when per-node memory grows large. EloqKV avoids these pitfalls:

- **Zero-fork checkpointing** incrementally flushes only dirty entries, eliminating costly memory fork operations.
- **Predictable long-tail latency**: checkpointing and compaction are carefully scheduled to avoid impacting the P99–P9999 latency profile.
- **No practical per-node memory ceiling**: EloqKV comfortably supports nodes with **50 GB+** of DRAM while safely managing multi-TB datasets on SSDs.
- **Robust replica join and recovery**: standby nodes join without overwhelming primaries, avoiding the "rejoin storm" patterns of large Redis clusters (write-buffer OOM → rejoin → fail).

Simplified Disaster Recovery and Instant Branching

EloqKV is built for cloud-native resilience and rapid iteration:

- **Disaster Recovery (DR) via object storage**: periodic snapshots are streamed directly to cloud object storage with cross-region replication. This avoids the need for auxiliary systems like Redis Streams or Kafka for log shipping, and keeps standby regions nearly cost-free until failover.
- **Branching in seconds**: teams can create isolated, fully functional database branches from object storage snapshots using **zero-copy cloning**. This enables realistic staging, load testing, and experimentation even on **100 TB+** datasets without duplicating physical storage.

Performance Benchmarks

EloqKV has been extensively benchmarked on NVMe SSD infrastructure to validate latency and throughput under realistic load.

- **Hardware profile**: Tests were run on Google Cloud `z3-highmem-16` instances with **16 vCPUs, 128 GB RAM**, and **2.9 TB NVMe SSD × 2**.
- **High throughput at DRAM and SSD scale**: Across dataset sizes from **20 GB (DRAM-sized)** to **200 GB–2 TB (SSD-sized)**, EloqKV sustained **~600K QPS** for DRAM-resident access and **~300K QPS** when serving purely from disk.
- **Consistently low long-tail latency**: On a **2 TB** dataset (~800 million keys, 2.5 KB average value size) with only **100 GB of DRAM** and **100K QPS**, EloqKV maintained **P99.99 latency between 1.5 ms and 3.1 ms**, depending on the read/write mix.
- **Linear horizontal scale-out**: Throughput scales roughly linearly with node count (e.g., 10× nodes ≈ 10× throughput). EloqKV remains **compatible with Redis smart clients** and supports client-side routing across shards.

Hardware and Software Setup

The benchmark configuration is summarized below:

Service type	Node type	Node count	Disk configuration
EloqKV	z3-highmem-16	1	2.9 TB × 1 NVMe
Redis	z3-highmem-16	1	2.9 TB × 1 NVMe
KvRocks	z3-highmem-16	1	2.9 TB × 1 NVMe

Benchmark Methodology

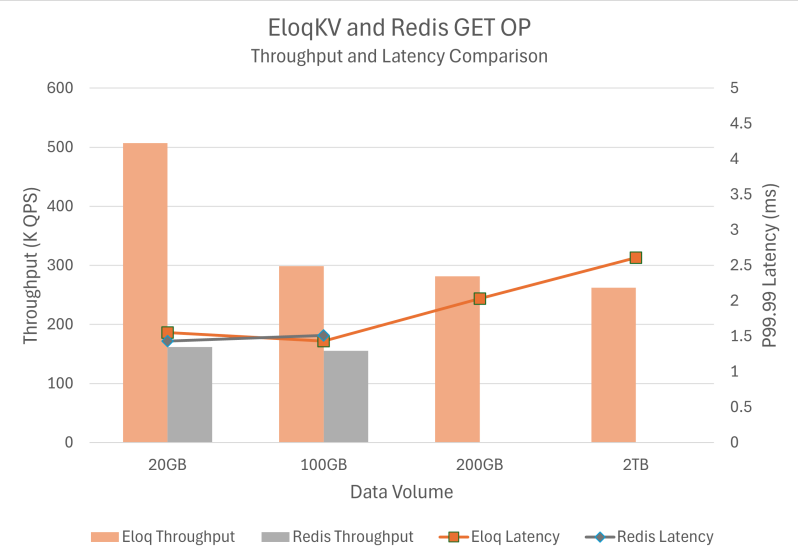
All benchmarks used `memtier_benchmark` with the following representative commands:

```
# load data
memtier_benchmark -t 10 -c 5 -s $SERVER_PRIVATE_IP -p 6379 -n allkeys --
distinct-client-seed --ratio=1:0 --key-prefix="kvkeyprefix_" --key-
minimum=1 --key-maximum=2000000000 --random-data --data-size=1000 --hide-
histogram --key-pattern=P:P
# query data
memtier_benchmark -t 10 -c 5 -s $SERVER_PRIVATE_IP -p 6379 --distinct-
client-seed --ratio=5:95 --key-prefix="kvkeyprefix_" --key-minimum=1 --
key-maximum=2000000000 --random-data --data-size=1000 --hide-histogram --
print-percentiles=50,99,99.9,99.99 --test-time=360 --randomize --rate-
limit=2000
```

We ran multiple workloads to compare EloqKV against Redis and other SSD-capable systems.

Results

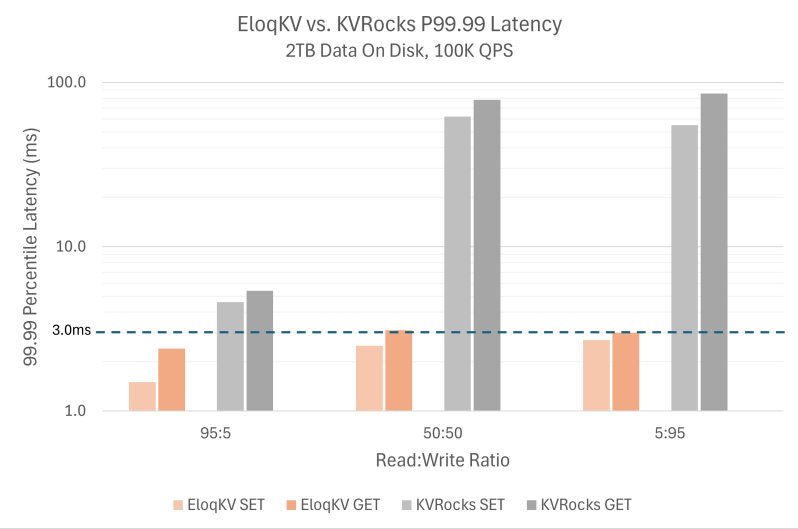
Benchmark 1 – EloqKV vs. Redis across dataset sizes (throughput and P99.99 latency):



Traditional DRAM-centric systems such as Redis are fundamentally limited by memory capacity. As dataset size approaches node memory, operators are forced to either overprovision DRAM or shard aggressively. In our tests, Redis delivered good performance at small sizes but could not scale beyond memory capacity, while EloqKV continued to operate smoothly by leveraging NVMe.

At **20 GB** and **100 GB**, EloqKV delivered **higher throughput than Redis** with a nearly identical P99.99 latency profile. Once the dataset exceeded memory capacity, Redis failed to handle the workload, whereas EloqKV scaled out to **2 TB** while keeping P99.99 latency stable at a few milliseconds—performance previously achievable only with DRAM-only architectures.

Benchmark 2 – EloqKV vs. Apache KvRocks (long-tail latency across workloads):



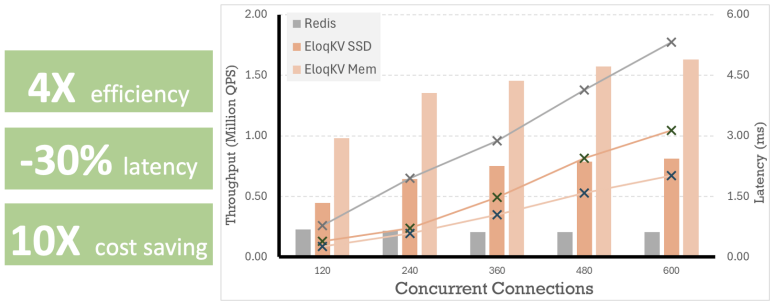
We also compared EloqKV to Apache KvRocks, another SSD-serving key–value store. Under IO-intensive workloads, KvRocks exhibited rapidly increasing P99.99 latency as background operations interfered with foreground reads (note the log scale on the chart). EloqKV, by contrast, kept P99.99 latency bounded and predictable.

To host a **2 TB** dataset with Redis, organizations typically deploy **~20 nodes** to get enough aggregate DRAM. EloqKV achieves comparable or better long-tail latency on a **single NVMe-optimized node**, reducing infrastructure cost by up to **20x** while simplifying operations substantially.

Customer Use Cases

1. AI Chat History for a Leading Smartphone Manufacturer

- **Scenario:** A top global smartphone manufacturer powers AI chat assistants across hundreds of millions of devices. Unlike traditional human-to-human chat, AI conversations generate much larger messages that bundle prompts, system context, and model outputs. Over time, this dramatically increases the volume of chat history that must be stored and retrieved in real time.
- **Challenge with previous solution:** The customer previously ran this workload on **AWS ElastiCache for Redis**. Because Redis keeps the full dataset in DRAM, the growing prompt and message history drove cluster memory requirements—and therefore cost—up sharply.
- **EloqKV deployment:** By migrating to EloqKV, the customer kept hot conversation state in memory while safely tiering the rest of the history to NVMe SSDs, all behind a Redis-compatible interface.
- **Outcome:** The customer achieved approximately **10x cost reduction** compared to their previous ElastiCache deployment, while maintaining the low-latency experience required for interactive AI chat.



2. Feature Store for Large-Scale E-Commerce Recommendations

- **Scenario:** A major e-commerce platform maintains a feature store containing **billions of product SKUs** and associated features (pricing, inventory, embeddings, and user–item interaction signals). Each recommendation request needs to fetch feature vectors for **up to 1,000 SKUs in a single call**, with a strict **P99 latency SLA of < 20 ms**.
- **Challenge with previous solution:** The feature store was previously implemented on Redis. To meet latency targets, the team stored the entire feature set in DRAM across a large Redis cluster. This resulted in very high memory costs and operational complexity as product and feature counts grew.
- **EloqKV deployment:** The company migrated the feature store to EloqKV, using Redis-compatible data structures for fast scatter–gather access patterns. Hot features remain in DRAM, while the much larger tail of SKUs is served directly from NVMe SSDs without violating latency SLAs.
- **Outcome:** EloqKV **reduced DRAM usage and cluster size by roughly 10x** while continuing to meet the **P99 < 20 ms** requirement for 1,000-SKU fetches. This enabled the team to scale the recommendation system economically as catalog size and model complexity increased.

3. Social Graph Storage for a Large Social Media Platform

- **Scenario:** A social media company stores user friendship and follow relationships for hundreds of millions of users. The social graph currently occupies around **500 GB** and is projected to grow to **1 TB+**. Friendship data is accessed extremely frequently whenever a user is online—for feed generation, mutual-friend suggestions, and safety checks.
- **Challenges with previous solutions:**

    - With **Redis**, all online and offline friendship edges had to be kept fully in memory. This was expensive and still required a separate MySQL cluster for persistence and recovery.
    - The company later migrated to **Amazon DynamoDB**, which provided durable storage at lower day-one cost. However, as user engagement and QPS grew beyond **~30K requests per second**, the combination of read/write throughput charges and autoscaling behavior caused costs to climb steeply—eventually exceeding the projected Redis TCO at higher traffic levels.
- **EloqKV deployment:** By adopting EloqKV as the primary store for the friendship graph, the company consolidated caching and persistence into a single Redis-compatible system. Hot graph neighborhoods are served from DRAM, while the rest of the graph is efficiently stored on NVMe SSDs with database-grade durability.
- **Outcome:** The platform achieved **around 10x cost savings** compared to the projected cost of scaling either Redis or DynamoDB for this workload, while still delivering the low-latency graph lookups required for real-time social experiences at peak traffic.